

3B – User Stories

Modul: M324 – DevOps-Prozesse mit Tools unterstützen

Autor: Metehan Celik

Datum: 22.05.2026

Gemeinsames Projekt erstellen

Im Rahmen der Aufgabe wurde das bestehende Basisprojekt (ToDo-App mit Spring Boot Backend und React Frontend) als Grundlage verwendet. Das Repository wurde auf GitHub initialisiert und alle relevanten Projektdateien wurden committed und gepusht.

Bild einfügen

User Stories

Die drei User Stories zu diesem Projekt wurden im Rahmen der Aufgaben 2B und 2C bereits zuvor erstellt und als Issues auf GitHub erfasst.

Bild einfügen

Labels

Zur besseren Organisation wurden Labels verwendet, um Issues zu kategorisieren. Das Label-Schema unterscheidet zwischen der Art der Anforderung und dem Status eines Issues:

Label	Bedeutung
user story	Kennzeichnet ein Issue als User Story
task	Kennzeichnet ein Issue als konkrete Teilaufgabe
enhancement	Neue Funktion oder Erweiterung
bug	Fehler im bestehenden Code
documentation	Verbesserungen oder Ergänzungen zur Dokumentation

Bild einfügen

Meilensteine

Für jede User Story wurde ein eigener Milestone erstellt. Dadurch lässt sich der Fortschritt jederzeit verfolgen: GitHub zeigt bei jedem Milestone an, wie viele der zugehörigen Issues bereits geschlossen wurden.

Die drei Milestones:

- **User Story 1 erledigt** – Persistenz mit Datenbank
- **User Story 2 erledigt** – Todos bearbeiten
- **User Story 3 erledigt** – Fälligkeitsdatum und Sortierung

Bild einfügen

Tasks und Issues

Die Teilaufgaben, welche zur Erfüllung der User Stories erledigt werden müssen, wurden als separate Issues angelegt und den jeweiligen User Stories in der Issue-Beschreibung als Checkliste verknüpft.

Bild einfügen

Projektmanagement und Kanban

Für den agilen Überblick des Projektfortschritts wurde ein GitHub Kanban-Board erstellt. Die User Stories und ihre Tasks werden dort in den Spalten **Backlog**, **Ready**, **In Progress** und **In Review** aufgeführt und bei Fortschritt entsprechend verschoben.

Bild einfügen

Zuweisung der Issues

Da es sich um ein Einzelprojekt handelt, wurden alle Issues dem einzigen Contributor zugewiesen. Jedes Issue ist mit dem passenden Label, Milestone und einer kurzen Beschreibung versehen.

Bild einfügen

Branching Strategie

Für jedes Issue wird ein eigener Feature-Branch erstellt. Das Namensschema lautet:

```
feature/issue-<nummer>-<kurzbeschreibung>
```

Beispiel:

```
feature/issue-22-jpa-entity  
feature/issue-26-put-endpoint
```

Die User Stories und Tasks werden mit dem jeweiligen Branch verknüpft, indem die Issue-Nummer im Commit-Message referenziert wird. Nach Abschluss der Arbeit wird der Branch via Pull Request in **main** gemergt.

Bild einfügen

Commits und Pull Requests

Nach Abschluss einer Aufgabe werden die Änderungen mittels Pull Request eingereicht. Der Pull Request wird mit dem passenden Milestone und den zugehörigen Issues verknüpft. Issues werden automatisch geschlossen, wenn im PR `Closes #<nummer>` verwendet wird.

Bild einfügen

Beschreibung von drei Commits:

- **feat: add H2 dependency to pom.xml**
Erweiterung der Maven-Konfiguration um die H2-Datenbank-Abhängigkeit, damit Todos persistent gespeichert werden können.
- **feat: annotate Task entity with JPA**
Anpassung der Task-Klasse mit JPA-Annotationen (@Entity, @Id), damit sie von Spring Data als Datenbankentität erkannt wird.
- **feat: replace ArrayList with TaskRepository**
Ersatz der In-Memory-Liste durch ein JPA-Repository, sodass alle Änderungen direkt in der Datenbank persistiert werden.